Politecnico di Milano – Sede di Cremona

Ingegneria del Software – a. a. 2006/07

Appello del 18 Settembre 2007

Cognome

Nome

Matricola

Istruzioni

1. La mancata indicazione dei dati anagrafici comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 1h30.
6. Punteggio totale a disposizione: 25/30.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Totale .....

Esercizio 1 (punti 6)

L'astrazione procedurale *scambia* riceve in ingresso due array a e b, che contengono valori interi. Sotto l'ipotesi che i due array abbiano la stessa dimensione, l'astrazione scambia fra loro il contenuto dei due array. Si completi la seguente specifica formale in JML dell'astrazione.

```
//@ requires
a!=null && b!= null && a.length == b.length;
.....................................................................
//@ ensures .....................................................
(\forall int i; 0<=i && i<a.length;
a[i]==\old(b[i]) && b[i]== \old(a[i]);
.....................................................................
.....................................................................
.....................................................................
.....................................................

public static void scambia (int [] a, int [] b)
```

Esercizio 2 (punti 12)
Si consideri la seguente specifica dell'ADT BranoMusicale:

public /*@ pure */@class BranoMusicale {
//OVERVIEW: tipo immutabile che rappresenta un brano musicale registrabile su supporti digitali
//Un tipico BranoMusicale è una quadrupla [info – durata – musica – bitrate], dove "info" è una stringa
//di informazioni testuali che descrivono il brano (per es. autore o titolo), "durata" è la durata in secondi
//del brano, "musica" è la rappresentazione digitale (in byte) della musica del brano e "bitRate è
//il numero di Kbit al secondo.

```
//costruttore:
//@ensures(*crea un nuovo brano sulla base dei valori passati come parametri*);
public BranoMusicale(String info, int durata, byte[] musica, int bitrate);

//metodi observer:
//@ensures(*restituisce le informazioni testuali associate al brano*);
public String getInfo();

//@ensures(*restituisce la durata in secondi del brano*);
public int getDurata();

//@ensures(*restituisce la rappresentazione digitale della musica del brano*);
public byte[] getMusica();

//@ensures(*restituisce il numero di bit al secondo della codifica del brano*);
public int bitRate();
```

}

a) Completare la parti con i puntini della seguente classe Memoria, che rappresenta una memoria per brani musicali (es. CD, MemoryStick ecc.)
In particolare sono da completare alcune specifiche JML di pre e postcondizioni, l'invariante di rappresentazione, l'invariante pubblico e il codice dei metodi "brani" e "inserisciBrano".

public class Memoria {
//OVERVIEW: Tipo mutabile. Una Memoria contiene tracce musicali di tipo BranoMusicale
//Una tipica Memoria è identificata dalla tripla [capienza, brani, durata], dove;
// "capienza" è la capacità in byte della memoria,
//"brani" è una sequenza <b1 b2 … bn> di oggetti della classe BranoMusicale,
//"durata" è la durata complessiva in secondi dell'insieme di brani presenti nella Memoria.

```
//@public invariant.....….
durata()== (\sumof int i; 0<=i && i<brani<().size(); brani().get(i).getDurata()) &&
capienza()>= (\sumof int i; 0<=i && i<brani<().size(); brani().get(i).getMusica().length) ….
.....….
.....….

//rep:
private int capienza;
private Vector<BranoMusicale> brani;
private int durata;
//@ private invariant .....….
capienza() == this.capienza && capienza>=0 && brani!=null &&
brani.length == brani().size() &&
(\forall i; 0<=i && i<brani().size(); brani().get(i)==brani[i])) ;
(NB: si suppone una relazione uno a uno fra la posizione nell'array e la posizione nel Vector)

.....….
.....….
```

```
        //@requires max_capienza>0;
        //@ensures (* crea una nuova Memoria di capienza specificata, inizialmente vuota*)
        public Memoria(int max_capienza){
                this.capienza=max_capienza;
                this.brani=new Vector();
                this.durata=0;
        }

        //@ensures (* /result è la durata complessiva dei brani in this *);
        public int durata() { return this.durata;}

        //@ensures (* /result è la capienza di this*);
        public int capienza() { return this.capienza;}

        //@ensures (* /result contiene tutti e soli i brani di this *);
        public Vector<BranoMusicale> brani() {
        //implementazione:
                return brani().clone();…
                ……
                ……
                ……
        //@ensures(*/result è true  se nella Memoria this c'è ancora abbastanza spazio per il brano b*)
        public boolean spazioDisponibile(BranoMusicale b) {
        //implementazione:
                int somma =0;
                (for int i; i>0 && i< brani.lenght;  i++)
                        somma+=brani.getMusica().length;.
                return (b.getMusica().length>=capienza -somma);.
                ……
                ……
        }

        //      se b supera la capacità ancora disponibile nella Memoria, solleva
        //      BranoTroppoLungoException. Altrimenti,  inserisce il brano b in this
        //@ensures ……
                capienza()>= b.getMusica().length +
                        \old((\sumof int i; 0<=i && i<brani().size(); brani().get(i).getMusica().length)) &&
                brani().size() == \old(brani.size()+1) &&
                b== brani().lastElement() &&
                (\forall int i; 0<=i && i<brani().size()-1; brani().get(i) == \old(brani().get(i)))
                (NB: si suppone che brano sia inserito in ultima posizione)
        //@signals( BranoTroppoLungoException e) capienza()<= b.getMusica().length +
                \old((\sumof int i; 0<=i && i<brani().size(); brani().get(i).getMusica().length))
        && brani().equals(\old(brani());

        public void inserisciBrano(BranoMusicale b) throws BranoTroppoLungoException {
        //implementazione:
                if (!spazioDisponibile() throw new BranoTroppoLungoException();
                brani.add(b);
                }
}
```

b) Si consideri la classe BranoTroppoLungoException utilizzata dalla classe Memoria. Sulla base dell'informazione disponibile, sarebbe più opportuno definirla come eccezione checked (sottoclasse di Exception) o eccezione unchecked (sottoclasse di RunTimeException)? Motivare brevemente la risposta.

Trattandosi di eccezione rilanciata allo scopo di avvertire il chiamante dell'impossibilita' di introdurre il brano in memoria, deve senz'altro essere checked (per costrignere il chiamante a "trattare" questo caso).

Esercizio 3 (punti 7)

a) Con riferimento all'esercizio 2, si consideri la seguente specifica:

```
public class BranoMP3-160Kbit extends BranoMusicale {
//OVERVIEW: BranoMusicale codificato in MP3 a 160 Kbit/sec.
//@also
//@public invariant .....................................................
        this.bitRate() ==160);

        //@ensures(*crea un nuovo brano sulla base dei valori passati come parametri*);
        public BranoMP3-160Kbit(String info, int durata, byte[] musica);
}
```

Si completi il public invariant e si indichi, motivando accuratamente ma sinteticamente la risposta, se se questo ADT soddisfa o meno il principio di sostituzione di Liskov rispetto a BranoMusicale.

L'ADT soddisfa il principio di sostituzione: la regola dei metodi non può essere violata da un costruttore (in quanto i costruttori non sono ereditati). Non e' violata nemmeno la regola delle proprietà, perche' un generico Brano può essere comunque da 160Kbit/sec.

b) Sempre con riferimento all'esercizio 2, si consideri la seguente specifica:

```
public class MP3Memory extends Memoria {
//OVERVIEW: Una memoria contenente solo brani di tipo BranoMP3-160Kbit.
//@also
//@public invariant .....................................................
(\forall int i; 0<=i && i<brani().size(); brani().get(i) instanceof BranoMP3-160Kbit);

        //@requires max_capienza>0;
        //@ensures (* crea una nuova MP3Memory, di capienza specificata, inizialmente vuota*)
        public MP3Memory(int max_capienza);
}
```

Si completi il public invariant e si indichi, motivando accuratamente ma sinteticamente la risposta, se questo ADT soddisfa o meno il principio di sostituzione di Liskov rispetto a Memoria.

L'ADT (un contenitore) non soddisfa il principio di sostituzione, in quanto il suo invariante impedisce di fatto l'inserimento di Brani che non siano di tipo BranoMP3-160Kbit. Per soddisfare l'overview (e quindi l'invariante) occorrerebbe ridefinire la specifica del metodo inserisciBrano, per impedire l'inserimento di brani con bitrate diverso da 160. Questo violerebbe la regola della postcondizione.